YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design

On

Anomaly Detection for Finance Transactions

Student Name: Salima M K
Register No: 23BSAMD027
16/04/2026

Industry Mentor: Sumit Shukla
Guided By: : Ankitha Poojary

TABLE OF CONTENTS

1. Introduction 3
   1.1 Scope of the Document 3
   1.2 Intended Audience 3
   1.3 System Overview 4
2. System Design 4
   2.1 Application Design 5
   2.2 Process Flow 6
   2.3 Information Flow 7
   2.4 Component Design 7
   2.5 Key Design Considerations 8
   2.6 Technology Stack 8
3. Data Design 9
   3.1 Dataset Description 9
   3.2 Data Access Mechanism 10
   3.3 Data Retention Policy 10
   3.4 Data Migration 10
4. Interfaces 10
5. State and Session Management 11
6. Caching 11
7. Non-Functional Requirements 12
8. References 13

1. INTRODUCTION

The rapid growth of digital financial systems has significantly increased the number of online transactions carried out every second. While this development has improved convenience and speed, it has also introduced serious concerns regarding fraudulent and abnormal financial activities. Detecting such anomalies is a critical requirement in modern financial systems to ensure security, trust, and reliability.

This project focuses on building a simple yet effective anomaly detection system for financial transactions using statistical techniques and machine learning concepts. The system is based on the Z-Score method, a widely used statistical approach that measures how far a data point deviates from the mean of a dataset in terms of standard deviations.

In this system, each transaction amount is analyzed against the overall distribution of transaction values. If a transaction has a Z-Score greater than a defined threshold (commonly ±3), it is considered abnormal and potentially fraudulent. This approach provides a simple yet powerful way to identify outliers without requiring complex deep learning models.

The system is implemented using Python and key data science libraries such as Pandas, NumPy, and Scikit-learn. To make the solution interactive and user-friendly, a Streamlit-based web application is developed, allowing users to upload datasets and instantly visualize detected anomalies through tables and graphs.

1.1 SCOPE OF THE DOCUMENT

This document provides a detailed high-level overview of the anomaly detection system. It describes the system architecture, data flow, design components, user interface, and technologies used. It also explains how statistical methods are applied to detect anomalies and how the results are presented through a web-based interface.

1.2 INTENDED AUDIENCE

This document is intended for students, academic evaluators, and developers who are interested in understanding the design and implementation of machine learning-based anomaly detection systems. It is also useful for beginners in data science who want to learn how statistical methods can be applied in real-world financial applications.

1.3 SYSTEM OVERVIEW

The system is designed to accept financial transaction data in CSV format from the user. Once the dataset is uploaded through the Streamlit interface, the backend processes the data using Pandas for cleaning and preprocessing.

The system then calculates the mean and standard deviation of transaction amounts and applies the Z-Score formula to each record. Based on the computed Z-Score, transactions are classified as either normal or anomalous.

The final output is displayed on a Streamlit dashboard, where users can view flagged transactions along with visual graphs that highlight unusual behavior patterns in the dataset.

2. SYSTEM DESIGN

2.1 APPLICATION DESIGN

The architecture of the system follows a simple layered design consisting of the user interface, backend processing, data processing module, and visualization layer.

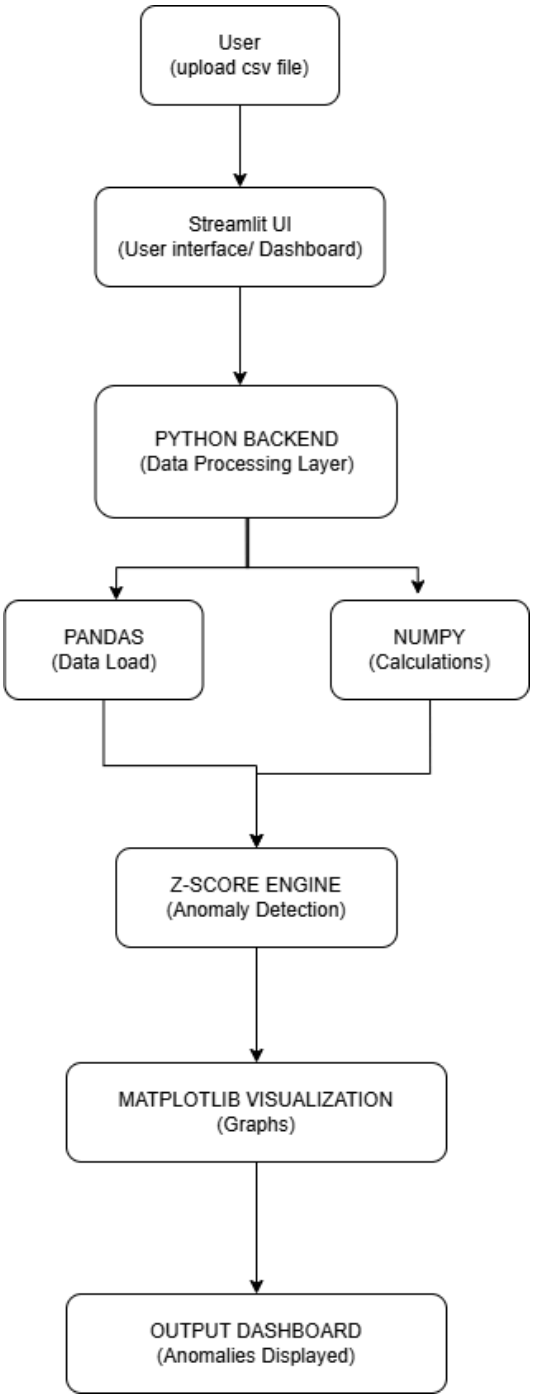

Figure 1 – Architecture of Anomaly Detection System

The user interacts with the system through a Streamlit web application, where they upload the transaction dataset. The request is passed to the backend Python engine, which processes the data using Pandas and NumPy.

The anomaly detection logic is implemented in the backend, where statistical calculations are performed. The processed results are then sent back to the frontend for visualization.

The visualization module displays results in the form of tables and graphs, helping users easily identify unusual transactions.

2.2 PROCESS FLOW

The process begins when the user uploads a CSV file through the Streamlit interface. The system reads the dataset and performs necessary preprocessing steps such as handling missing values and selecting relevant features.

After preprocessing, the system calculates the mean and standard deviation of the transaction amount column. Using these values, the Z-Score for each transaction is computed.

Once the Z-Scores are calculated, the system compares them with a predefined threshold. Transactions that exceed this threshold are marked as anomalies.

Finally, the results are displayed on the dashboard, where users can visually analyze normal and abnormal transactions.

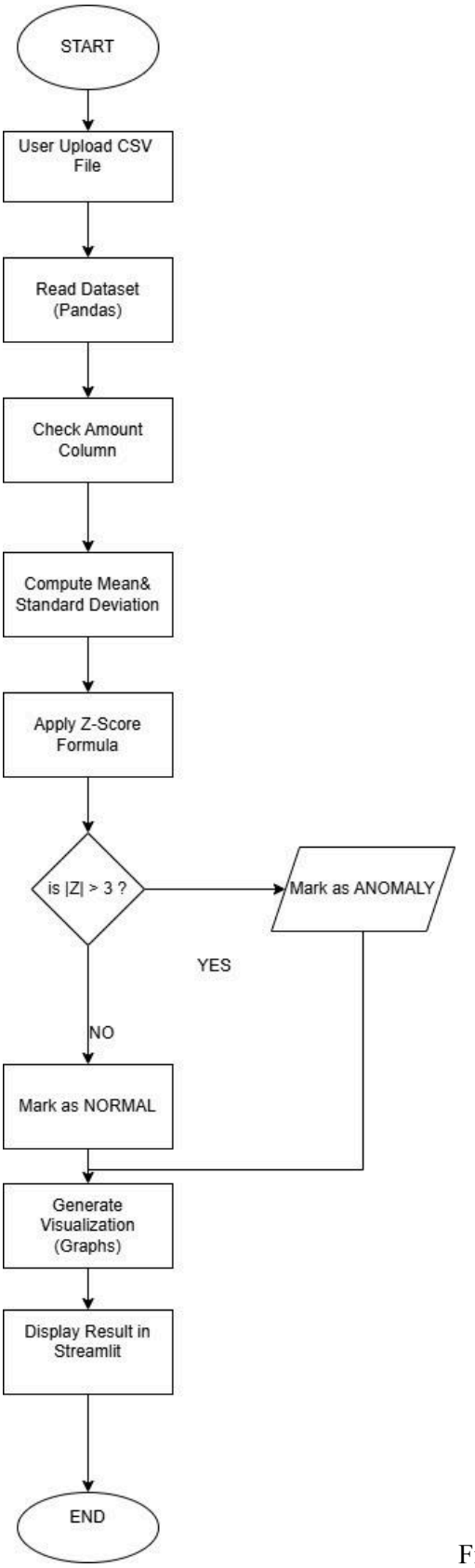

Figure 2 – Flow of Anomaly Detection Process

2.3 INFORMATION FLOW

The information in the system flows from the user interface to the backend processing layer. The uploaded dataset is transmitted to the Python backend, where it is processed and analyzed.

The anomaly detection engine receives cleaned data and computes statistical values. The results are then passed back to the Streamlit interface, where they are presented in a structured format.

User interaction and feedback remain within the session, ensuring a smooth and continuous workflow.

2.4 COMPONENT DESIGN

The system consists of four major components. The user interface component is responsible for handling file uploads and displaying results. The data processing component handles dataset cleaning and transformation.

The anomaly detection component applies statistical methods to identify outliers in the dataset. Finally, the visualization component generates graphs and charts to represent the results in a meaningful way.

2.5 KEY DESIGN CONSIDERATIONS

The system is designed with simplicity and performance in mind. It ensures fast processing of datasets using efficient Python libraries. The interface is kept minimal and user-friendly to allow easy interaction.

The model focuses on interpretability rather than complexity, making it suitable for academic and real-world understanding. Scalability is also considered so that larger datasets can be processed in future enhancements.

2.6 TECHNOLOGY STACK

The system is developed using Python as the core programming language. Pandas and NumPy are used for data processing and numerical computation. Scikit-learn supports statistical and anomaly detection operations.

Matplotlib is used for data visualization, and Streamlit is used to build the interactive web application interface.

3. DATA DESIGN

The system is based on structured financial transaction data stored in CSV format. Each record represents a transaction with multiple attributes including time, transaction amount, and anonymized features.

Column Name	Data Type	Description
Time	Numeric	Seconds elapsed between transactions
V1-V28	Float	PCA transformed features of transactions
Amount	Float	Transaction value
Class	Integer	0 = Normal transaction, 1 = Fraud/anomaly

The dataset typically includes columns such as Time, Amount, and Class. The Amount column is the primary attribute used for anomaly detection, while the Class column (if present) indicates whether a transaction is normal or fraudulent.

The data is processed in memory using Pandas without the need for an external database. This ensures fast computation and simplicity in design.

3.2 DATA ACCESS MECHANISM

Data is accessed through file upload in Streamlit. Once uploaded, Pandas reads the CSV file into a structured DataFrame, which is then used for processing and analysis. All computations are performed in memory.

3.3 DATA RETENTION POLICY

The system does not permanently store user data. Uploaded datasets are temporarily stored during the session and automatically cleared once the session ends.

3.4 DATA MIGRATION

Since the system uses standard CSV format, data can easily be migrated or reused in other machine learning pipelines or upgraded database systems without modification.

4. INTERFACES

The system uses a web-based interface developed using Streamlit. The interface allows users to upload datasets, view processed results, and analyze visual outputs.

Communication between the frontend and backend occurs through Python function calls within the Streamlit framework. This eliminates the need for complex API layers and ensures faster execution.

External interfaces are not heavily used, although future enhancements may include integration with financial APIs for real-time transaction monitoring.

5. STATE AND SESSION MANAGEMENT

The application utilizes the built-in session management capabilities of Streamlit to maintain the state of user interactions during runtime. Session state enables the system to preserve user inputs, uploaded datasets, and computed outputs across different components of the application without requiring repeated execution.

This ensures a seamless and interactive user experience, allowing efficient navigation between data upload, processing, and visualization stages. However, Streamlit follows a stateless execution model between sessions; therefore, all session-specific data is cleared automatically when the page is refreshed or the application is closed. This behavior enhances data privacy by ensuring that no user data is retained beyond the active session.

6. CACHING

To improve performance and reduce computational overhead, the application implements caching using Streamlit's caching mechanism (e.g., @st.cache_data). Caching allows the system to store the results of resource-intensive operations such as dataset loading and anomaly detection.

When the same dataset or computation is requested again, the cached result is returned instead of recomputing it. This significantly reduces execution time and improves responsiveness, especially when working with large-scale financial datasets.

The use of caching ensures efficient resource utilization while maintaining consistency and reproducibility of results within the application.

7. NON-FUNCTIONAL REQUIREMENTS

The system is designed to satisfy key non-functional requirements essential for a financial data analysis application:

Performance:
The system ensures efficient processing of transaction datasets using optimized data handling techniques provided by Pandas and NumPy. Caching further enhances performance by minimizing redundant computations.

Usability:
The user interface is simple, intuitive, and interactive, enabling users with minimal technical knowledge to easily upload datasets, perform anomaly detection, and interpret results.

Reliability:
The application produces consistent and accurate results based on statistically robust methods such as the Z-score (Standard Score), ensuring dependable anomaly detection.

Security and Privacy:
No sensitive data is permanently stored within the system. All data is processed temporarily in memory during the active session, reducing the risk of data leakage.

Scalability:
The system is designed to support future enhancements, including database integration, cloud deployment, and the addition of advanced machine learning models such as Isolation Forest or Local Outlier Factor for improved fraud detection.

8. REFERENCES

Standard Score (Z-score). Available at: https://en.wikipedia.org/wiki/Standard_score
Pandas Documentation. Available at: https://pandas.pydata.org/docs/
NumPy Documentation. Available at: https://numpy.org/doc/
Matplotlib Documentation. Available at: https://matplotlib.org/stable/contents.html
Streamlit Documentation. Available at: https://docs.streamlit.io/
Scikit-learn Outlier Detection. Available at:
https://scikit-learn.org/stable/modules/outlier_detection.html
Credit Card Fraud Detection Dataset (Kaggle). Available at:
https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
Anomaly Detection in Credit Card Transactions. Available at: https://arxiv.org/abs/1901.03407